Lab 2 – Assembly Programming, Signal Measurement, & Peripheral Interfacing on the mbed
Part 1: PWM & Timer Interrupt

Embedded Processors UESTC2004 & UESTCHN2004 School of Engineering, University of Glasgow

Student Name: Kartike Sharan

Student Number: 3008620s

UESTC 2004 Lab Assessment Criteria

Generally, each individual lab will be evaluated using the below criteria.

- Demonstrating a functional embedded system

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Lab Questions

- Reflection and significance

  After the completion of programming and testing each of the "Tasks", you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves "What you did?", "What it means?", and "What next?", meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice
  - Meaningful variable names – please avoid names like "x", "y", or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code/ does.

- Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice ✓

```
#include “mbed.h”
char switch_word ;                    //the word we will send char
recd_val;                             //the received value
int main() {
    async_port.baud(9600);                    //set baud rate to 9600
        recd_val = 0;
        while (1)
        {
            switch_word=0xa0;
        }
}
```

Avoid this ✗

```
#include “mbed.h”
char switch_word ;
char recd_val;
int main() {
async_port.baud(9600);
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

Lab Objective

- Write and execute assembly language programs to perform arithmetic operations using an ARM simulator, reinforcing low-level programming concepts.
- Understand register operations and instruction sets to see how they impact program execution in an ARM-based environment.
- Analyse embedded output signals using an oscilloscope to deduce the functionality of an unknown executable, including identifying PWM characteristics and digital timing behaviour.

- Demonstrate practical use of hardware timers and interrupts (using Ticker) to implement non-blocking periodic tasks in real-time embedded systems.
- Design and implement PWM-based control logic by configuring duty cycles and frequency on a microcontroller, and validate the output using measurement tools.

Requirements

Software:

- Keil arm online compiler (https://studio.keil.arm.com/auth/login/) – You will need to create an account to use the online compiler.
- Download and install the ARMSim#1.91 from Moodle-> Lab Material -> "Lab 2"(Folder).

  Note: To run ARMSim# 1.91 you must be running a **Windows system** with a **.NET** Framework. To run it on Linux or Mac OS, follow the instructions on its official website. **(Please do this before attending the lab)**

Task 1: Programming in Assembly

Task 1.1: ARMSim# is not an editor, so you will need to create and edit your ARM assembly program with a separate text editor (such as Notepad++). Open your favourite text editor and type in the following simple program:

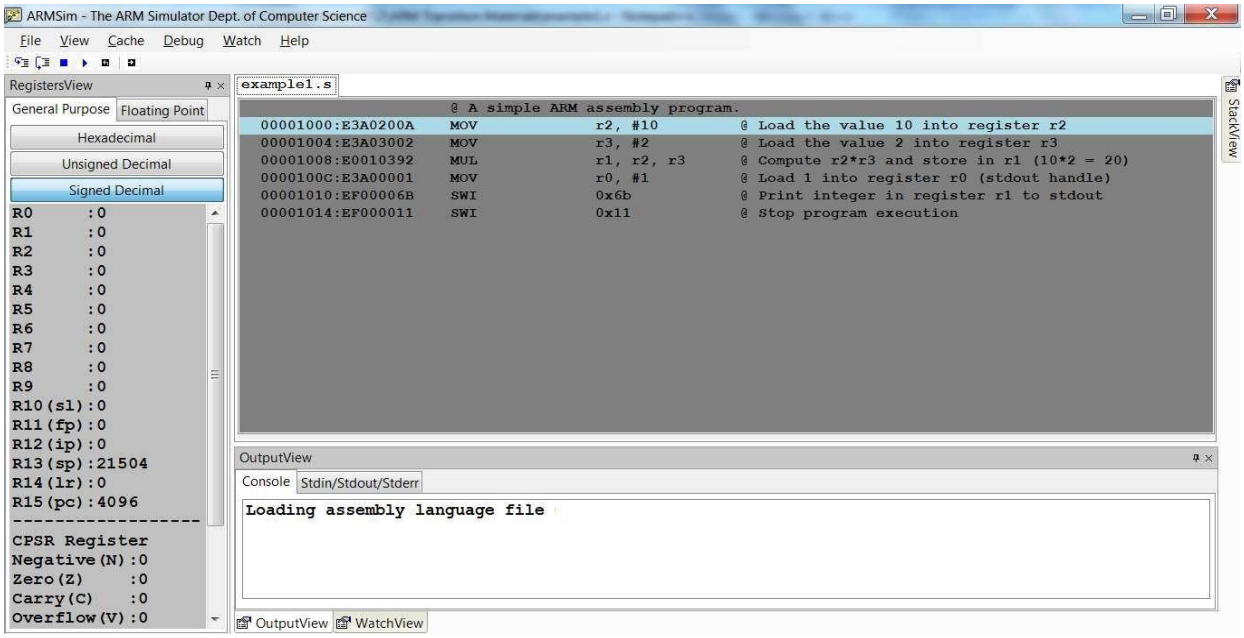
Do not worry if some of these instructions are unfamiliar at the moment, they will be covered in time. Save your program as example1.s – note the '.s' file extension is required in order for ARMSim# to read your file. Once you have saved your ARM program, you need to load it into ARMSim#. Open the ARMSim# program, select 'File -> Load' from the menu, and select your example1.s file. It should open your program and make sure it is without syntactic errors.
Your screen will look like the one below. Your example1.s file should look like below:

```
@ A simple ARM assembly program.
MOV r2, #10 @ Load value 10 into register
r2
MOV r3, #2 @ Load value 2 into register r3
MUL r1, r2, r3 @ Compute r2*r3 & store in r1(10*2 =
20)
MOV r0, #1 @ Load 1 into register r0 (stdout handle)
SWI 0x6b @ Print integer in register r1 to stdout
SWI 0x11 @ Stop program execution
```

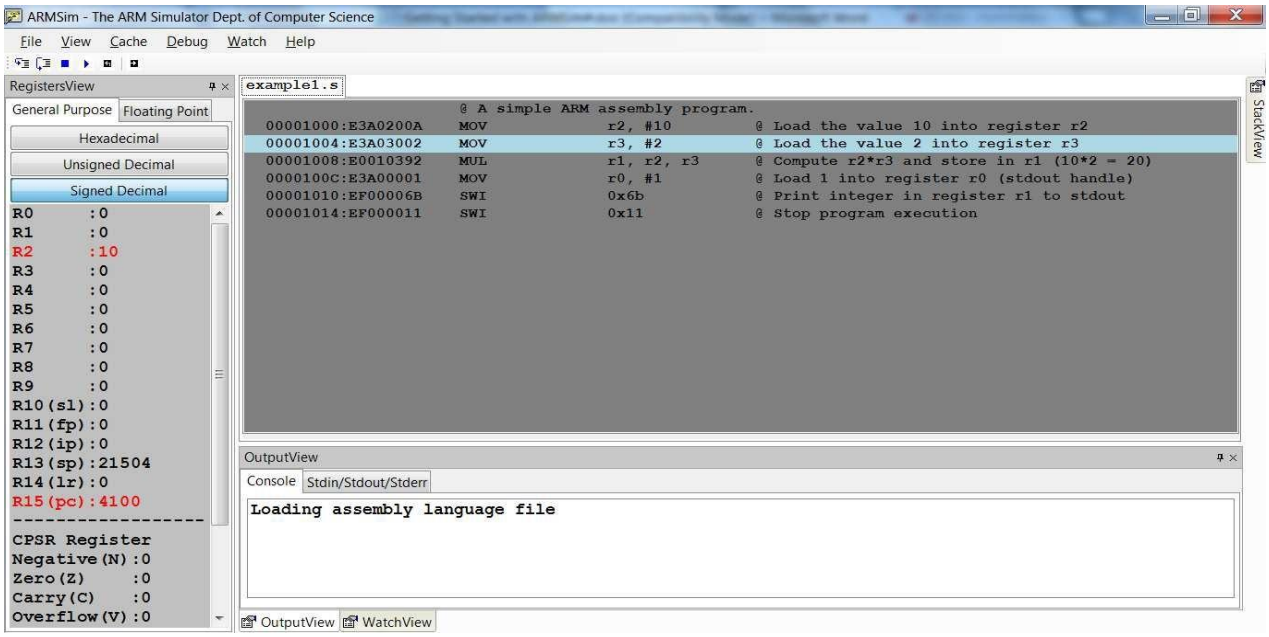
On the left side of the screen, the registers for the simulated ARM processor are displayed. The source code window is in the centre, while the console window is at the bottom.

There are three options for running (simulating) your assembly language program – run,

step into, and step over. The two that will likely be of most use (and their toolbar icons) are (i) run( ) and (ii) step into ( ). Run (keyboard shortcut F5) will run your assembly language program from start to finish, while step into (keyboard shortcut F11) will execute one instruction at a time, allowing you to see how each instruction is modifying the state.



Press “Step Into”, and you should see a screen like the one below.

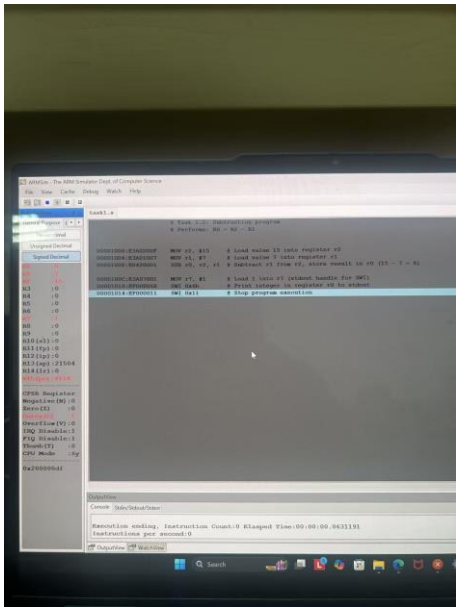


On the left hand side, you will see that the registers that have been changed by the executed instruction are red. In the source code window, you will notice that the highlighted blue line is now the next instruction – is the instruction pointed to by the program counter,

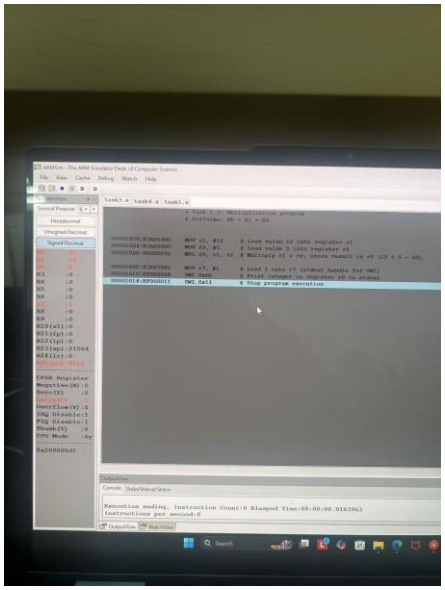
and will be executed next. Try stepping through the remainder of the simple program, verifying that the registers are changing as expected.

Task 1.2: Complete the following tasks with reference to the codes in Task 1.1

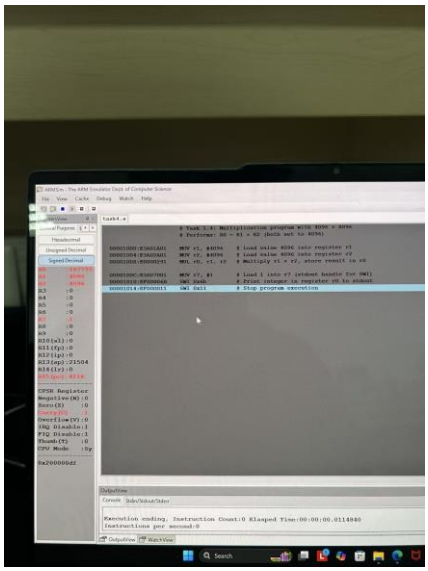
Write a program to perform the subtraction of a value in register 1 from a value in register 2 and the result should be stored in register 0. Once completed, show the demo to the GTA to get your mark.



Task 1.3: Write a program to multiply the value in register 1 with the value in register 2 and store the result in register 0.



Task 1.4: With the multiplication program, set registers 1 and 2 to 4096. What is the result from the simulator? **[Answer in the below space]**



Task 1.5: Debugging

The following are some debugging hints that you might consider if Task 1 output is not as expected:

- **Syntax Errors in Assembly:** Ensure proper instruction format and case sensitivity.
- **Incorrect Register Values:** Step through code using the "Step Into" function.

- **Unexpected Output:** Double-check operand order and register assignments.

Task 2: PWM & Interrupts on mbed L432KC

Context/Scenario:

You are an embedded systems engineer developing part of a control unit for an environmental monitoring device. The device uses a microcontroller to process analogue signals from sensors, and control output actuators via PWM. As part of prototyping, you need to analyse the behaviour of a compiled program running on the mbed L432KC board. You are not given the code, only the executable file which you can find on Moodle in the "Lab 2" folder called "***UESTC_UESTCHN2004_Lab2.bin***".

You are tasked with reverse-engineering the functionality of the binary and extending the functionality based on your analysis. What you will need:

1. L432KC mbed board
2. Jumper wires
3. Oscilloscope (Two Channels). Hint: When using the oscilloscope to measure an unknown signal consider switching the trigger mode to Normal as low-frequency signals may not be picked up by the oscilloscope's "AUTO" capture.

Task 2.1: Understanding the Digital Output

- Using an oscilloscope, measure the signal on pin D5. Paste a picture of the oscilloscope screen below.
    - What type of waveform do you observe? **[Answer below]**



Square Wave

    - What is the period and frequency of the signal? **[Answer below]**
      1ms and 1khz

    - Based on your measurements, how often does the signal toggle? **[Answer below]**

0.5ms

- Assume the signal on D5 is toggled in software. Estimate the time delay or loop timing used in the code. What instruction or function might be responsible for this?

  delay_us(500)

Task 2.2: Investigating the PWM Signal

- Connect an oscilloscope to **D6**, where a PWM signal is being generated.
    - Paste a picture of the output below.



    - What is the **period** of the PWM signal? **[Answer below]**

      2s

    - From your measurements, calculate the **PWM frequency** in Hz. **[Answer below and show your workings]**

      500mhz

- Based on your findings:
    - What do you think the **PWM pulse width value** is (in seconds)? **[Answer below and show your workings]**

      500ms

    - What percentage of the time is the signal HIGH (duty cycle)? **[Answer below and show your workings]**

      26%

    - Based on your answers above, write the lines of code used to generate the PWM signal. **[Answer below]**

```
while(1){
   D5 = 1;     // HIGH
   delay(500); // 500 ms
   D5 = 0;     // LOW
   delay(1500); // 1500 ms (2s total period)
}
```

Task 2.3: Code Function Deduction

- From your observations, describe what the program is likely doing in high-level terms. Consider:
  - What kind of output is being generated on D5 and why? **[Answer below]**

    At D5 there is a 50% duty signal being outputed this may be used to trigger a transistor to half the outout power to a motor

  - What is the purpose of toggling the digital pin in this context? **[Answer below]**

    The purpose is to half the output

Task 2.4: Extension Task – Using a Ticker Interrupt

- Having inferred what the code does, extend it by adding a Ticker to Toggle Between Two Duty Cycles (30% and 70%). The PWM frequency should remain the same as Tasks 2.1 to 2.3.

[Paste your code below]

- Validate your code by measuring the period, frequency, and duty cycle of the generated PWM signals using an oscilloscope. Paste pictures of your measurement and detail your answer to show how you measured/calculated each.

[Answer below]

Task 4: Reflection

After the completion of programming and testing of the tasks please reflect, in a few lines, on your understanding of the tasks and their significance. This is a chance for you to ask yourselves **"What you did?", "What it means?", "What next?"**, and/or How else, more efficiently, can this be done?, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. **Please, do not provide a summary of what you did as it will not be accepted.**

[Answer below]

Working with both assembly and real signals made the gap between code and hardware obvious. Small instruction choices directly affect timing and behaviour on pins. It also showed that relying on delays isn't ideal—using timers and interrupts is cleaner and more reliable for real embedded systems.

Lab Score

Marking Criteria	Student's Score
Task 1.2 (5%) Task 1.3 (5%) Task 1.4 (5%)	
Task 2.1 (10%) Task 2.2 (10%) Task 2.3 (10%) Task 2.4 (35%)	
Task 4 Reflection (5%)	
Adherence to coding good practice (15%)	